Building simple Deep Neural Networks

CSCI-P 556

ZORAN TIGANJ

Today

- Building simple Deep Neural Networks (continuing with Chapter 10)
  - Deep learning architectures for regression and classification problems.
  - Building a simple classifier using Keras
    - Functional API
    - Sequential API
    - Subclassing API
  - Fine-tuning hyperparameters

Regression with MLP

- If you want to predict a single value (e.g., the price of a house, given many of its features), then you just need a single output neuron: its output is the predicted value.
- For multivariate regression (i.e., to predict multiple values at once), you need one output neuron per output dimension.
  - For example, to locate the center of an object in an image, you need to predict 2D coordinates, so you ne ed two output neurons.

Typical regression MLP architecture

Hyperparameter	Typical value
# input neurons	One per input feature (e.g., 28 x 28 = 784 for MNIST)
# hidden layers	Depends on the problem, but typically 1 to 5
# neurons per hidden layer	Depends on the problem, but typically 10 to 100
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU, see Chapter 11)
Output activation	None, or ReLU/softplus (if positive outputs) or logistic/tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

Typical classification MLP architecture

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross entropy	Cross entropy	Cross entropy

Equation 4-22. Cross entropy cost function

$$J(\boldsymbol{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

Implementing MLPs with Keras

- Keras is a high-level Deep Learning API that allows you to easily build, train, evaluate, and execute all sorts of neural networks.
- Keras can be run from multiple Deep Learning libraries such as: TensorFlow, Microsoft Cognitive Toolkit (CNTK), and Theano. These libraries serve as a computational backend.
- Let's build some neural networks with Keras.

Building a simple classifier using Keras

```
>>> import tensorflow as tf
>>> from tensorflow import keras
>>> tf.__version__
'2.0.0'
>>> keras.__version__
'2.2.4-tf'
```

```
fashion_mnist = keras.datasets.fashion_mnist
(X_train_full, y_train_full), (X_test, y_test) =
fashion_mnist.load_data()
```

```
>>> X_train_full.shape
(60000, 28, 28)
>>> X_train_full.dtype
dtype('uint8')
```

Building a simple classifier using Keras

```
model = keras.models.Sequential()
model.add(keras.layers.Flatten(input_shape=[28, 28]))
model.add(keras.layers.Dense(300, activation="relu"))
model.add(keras.layers.Dense(100, activation="relu"))
model.add(keras.layers.Dense(10, activation="softmax"))
```

```
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 300)               235500
_________________________________________________________________
dense_1 (Dense)              (None, 100)               30100
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 266,610
Trainable params: 266,610
Non-trainable params: 0
_________________________________________________________________
```

Building a simple regression model using Keras

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()

X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full)

model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu",
input_shape=X_train.shape[1:]),
    keras.layers.Dense(1)
])
model.compile(loss="mean_squared_error", optimizer="sgd")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3] # pretend these are new instances
y_pred = model.predict(X_new)
```

Building Complex Models Using the Functional API

- As we saw, the Sequential API is quite easy to use.
- However, although Sequential models are extremely common, it is sometimes useful to build neural networks with more complex topologies, or with multiple inputs or outputs.
- For this purpose, Keras offers the Functional API.

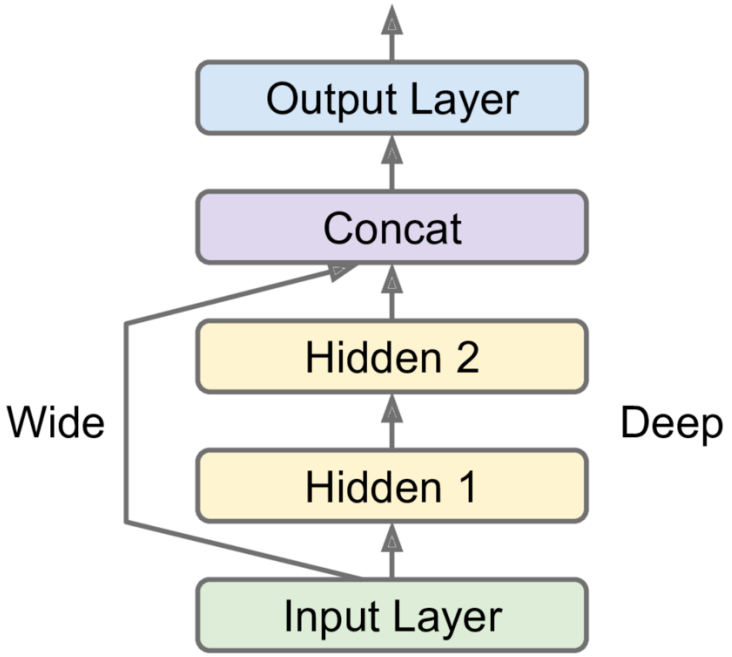

Figure 10-14. Wide & Deep neural network

This architecture makes it possible for the neural network to learn both deep patterns (using the deep path) and simple rules (through t he short path)

After Heng-Tze Cheng et al, 2016

Building Complex Models Using the Functional API

```
input_ = keras.layers.Input(shape=X_train.shape[1:])
hidden1 = keras.layers.Dense(30, activation="relu")(input_)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)
concat = keras.layers.Concatenate()([input_, hidden2])
output = keras.layers.Dense(1)(concat)
model = keras.Model(inputs=[input_], outputs=[output])
```

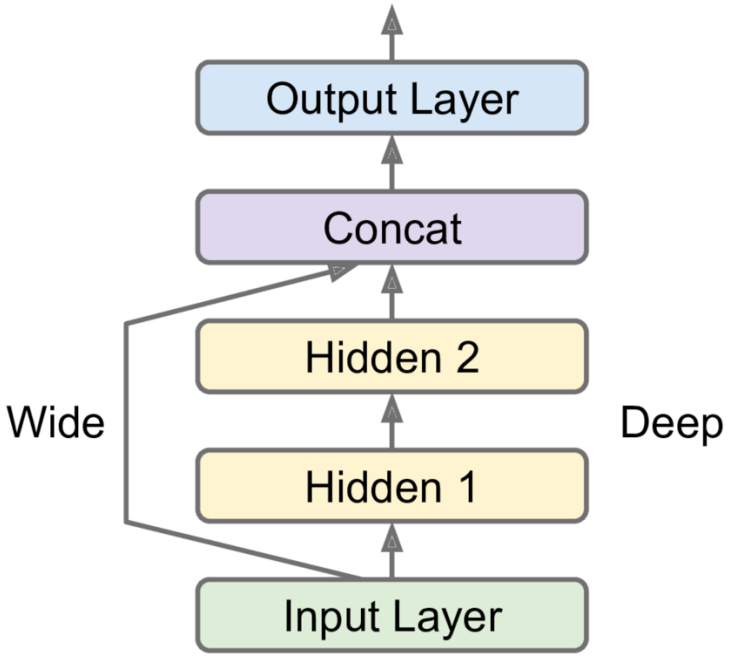

Figure 10-14. Wide & Deep neural network

This architecture makes it possible for the neural network to learn both deep patterns (using the deep path) and simple rules (through t he short path)

After Heng-Tze Cheng et al, 2016

Building Complex Models Using the Functional API

- What if you want to send a subset of the features through the wide path and a different subset (possibly overlapping) through the deep path

```
input_A = keras.layers.Input(shape=[5], name="wide_input")
input_B = keras.layers.Input(shape=[6], name="deep_input")
hidden1 = keras.layers.Dense(30, activation="relu")(input_B)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)
concat = keras.layers.concatenate([input_A, hidden2])
output = keras.layers.Dense(1, name="output")(concat)
model = keras.Model(inputs=[input_A, input_B], outputs=[output])
```

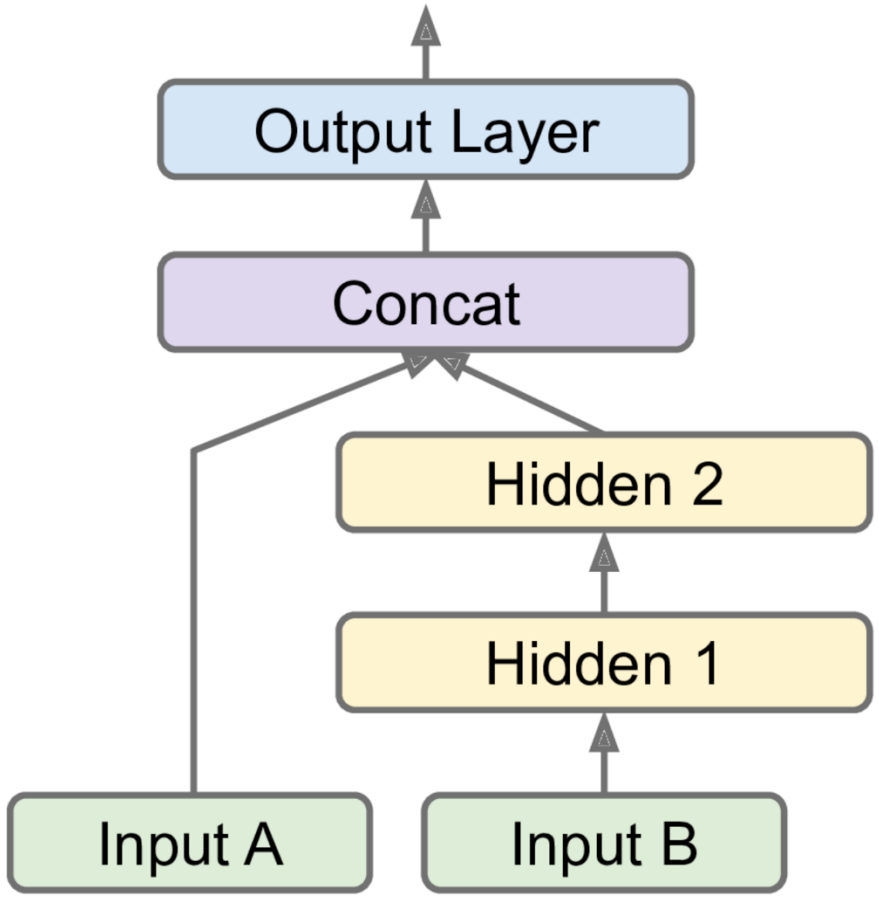

Figure 10-15. Handling multiple inputs

Building Complex Models Using the Functional API

- There are many use cases in which you may want to have multiple outputs:
  - You may want to locate and classify the main object in a picture.
  - Another use case is as a regularization technique. For example, you may want to add some auxiliary outputs in a neural network architecture to ensure that the underlying part of the network learns something useful on its own, without relying on the rest of the network.

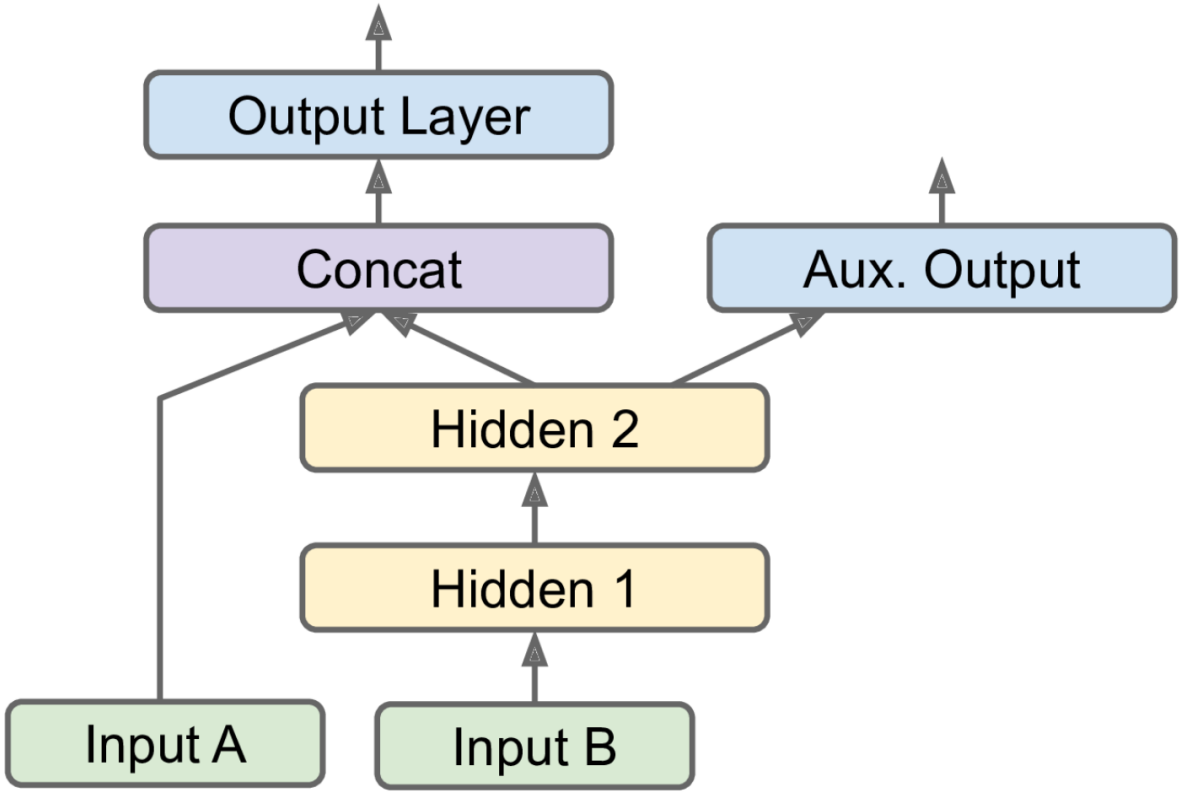

Figure 10-16. Handling multiple outputs, in this example to add an auxiliary output for regularization

Building Complex Models Using the Functional API

```
[...] # Same as above, up to the main output layer
output = keras.layers.Dense(1, name="main_output")(concat)
aux_output = keras.layers.Dense(1, name="aux_output")(hidden2)
model = keras.Model(inputs=[input_A, input_B], outputs=[output,
aux_output])
```

```
model.compile(loss=["mse", "mse"], loss_weights=[0.9, 0.1],
optimizer="sgd")
```

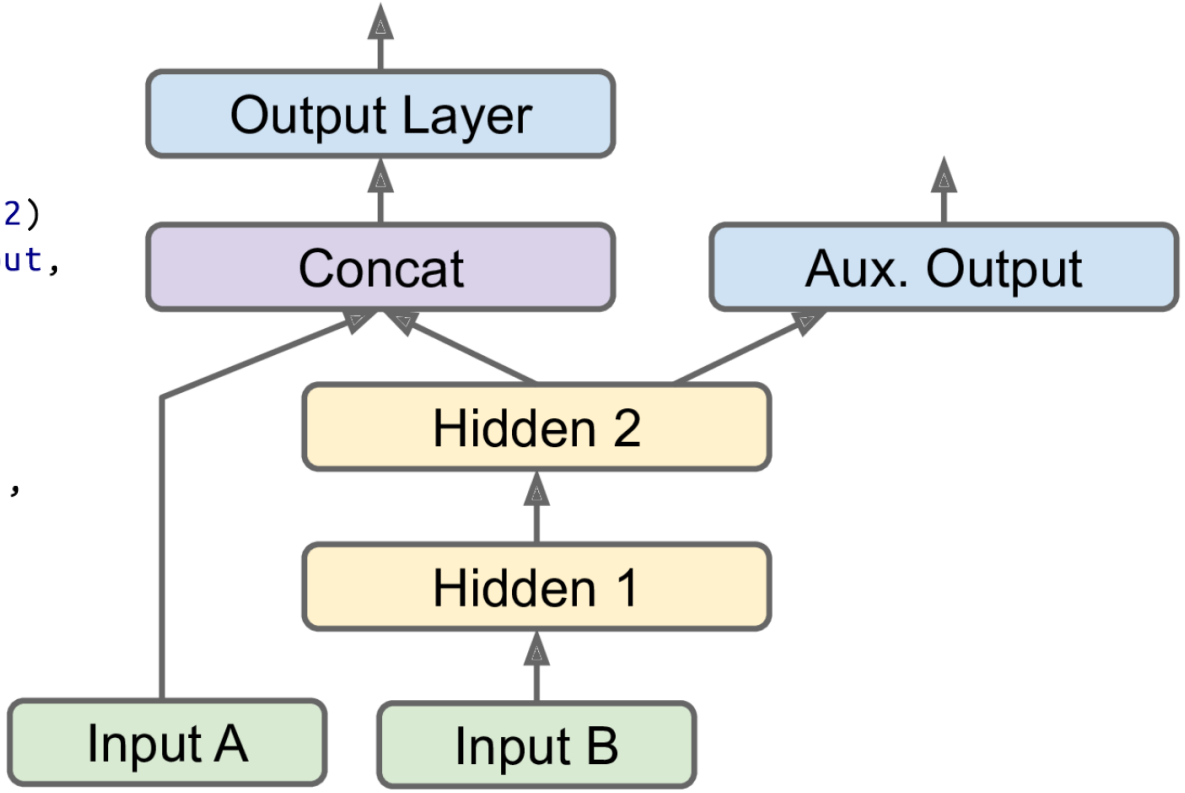

Figure 10-16. Handling multiple outputs, in this example to add an auxiliary output for regularization

Using the Subclassing API to Build Dynamic Models

- Both the Sequential API and the Functional API are declarative: you start by declaring which layers you want to use and how they should be connected, and only then can you start feeding the model some data for training or inference.
  - Easy to visualize, share, clone
- Some models involve loops, varying shapes, conditional branching, and other dynamic behaviors. For such cases, or simply if you prefer a more imperative programming style, the Subclassing API is for you.

Using the Subclassing API to Build Dynamic Models

- Simply subclass the Model class, create the layers you ne ed in the constructor, and use them to perform the computations you want in the call() method.

```
class WideAndDeepModel(keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs) # handles standard args (e.g., name)
        self.hidden1 = keras.layers.Dense(units, activation=activation)
        self.hidden2 = keras.layers.Dense(units, activation=activation)
        self.main_output = keras.layers.Dense(1)
        self.aux_output = keras.layers.Dense(1)

    def call(self, inputs):
        input_A, input_B = inputs
        hidden1 = self.hidden1(input_B)
        hidden2 = self.hidden2(hidden1)
        concat = keras.layers.concatenate([input_A, hidden2])
        main_output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return main_output, aux_output

model = WideAndDeepModel()
```

This code does the same as what we did before. The big difference is that you can do pretty much anything you want in the call() method: for loops, if statements, low-level TensorFlow operations—your imagination is the limit!

Using the Subclassing API to Build Dynamic Models

- This extra flexibility does come at a cost:
  - your model's architecture is hidden within the call() method, so Keras cannot easily inspect it; it cannot save or clone it;
  - when you call the summary() method, you only get a list of layers, without any information on how they are connected to each other.
  - Keras cannot check types and shapes ahead of time, and it is easier to make mistakes.
- So unless you really need that extra flexibility, you should probably stick to the Sequential API or the Functional API.

Saving and Restoring a Model

As simple as it gets:

```
model.save("my_keras_model", save_format="tf")
```

```
model = tf.keras.models.load_model("my_keras_model")
```

Fine-Tuning Neural Network Hyperparameters

- The flexibility of neural networks is also one of their main drawbacks: there are many hyperparameters to tweak.
- Not only can you us e any imaginable network architecture, but even in a simple MLP you can change
  - the number of layers,
  - the number of neurons per layer,
  - the type of activation function to use in each layer,
  - the weight initialization logic…

Fine-Tuning Neural Network Hyperparameters

- One option is to simply try many combinations of hyperparameters and see which one works best on the validation set (or use K-fold crossvalidation).
- For example, we can use GridSearchCV or RandomizedSearchCV to explore the hyperparameter space

```
from scipy.stats import reciprocal
from sklearn.model_selection import RandomizedSearchCV

param_distribs = {
    "n_hidden": [0, 1, 2, 3],
    "n_neurons": np.arange(1, 100),
    "learning_rate": reciprocal(3e-4, 3e-2),
}

rnd_search_cv = RandomizedSearchCV(keras_reg, param_distribs, n_iter=10,
cv=3)
rnd_search_cv.fit(X_train, y_train, epochs=100,
                  validation_data=(X_valid, y_valid),
                  callbacks=[keras.callbacks.EarlyStopping(patience=10)])
```

```
>>> rnd_search_cv.best_params_
{'learning_rate': 0.0033625641252688094, 'n_hidden': 2, 'n_neurons': 42}
>>> rnd_search_cv.best_score_
-0.3189529188278931
>>> model = rnd_search_cv.best_estimator_.model
```

Fine-Tuning Neural Network Hyperparameters

- Fortunately, there are many techniques to explore a search space much more efficiently than randomly.
- Their core idea is simple: when a region of the space turns out to be good, it should be explored more. Such techniques take care of the "zooming" process for you and lead to much better solutions in much less time.
- See book chapter 10 for info on some popular Python libraries you can use to optimize hyperparameters, e.g., Keras Tuner.

Tensorboard

- See book chapter 11 to learn more about callbacks and using Tensorboard for visualization.

Next time

- Custom Models and Training with TensorFlow (Chapter 12)
- Loading and Preprocessing Data with TensorFlow (chapter 13)
- CNNs (Chapter 14)